# Attention Is Not All You Need: Hierarchical WTA Circuits for Compositional Reasoning

Marwan Khaled Elgoghel[1]

[1]Monmouth University, Computer Science and Mathematics

April 2026

**Code and multi-seed results:** https://github.com/Elgoghel/hwta-circuits

**Abstract**

We propose a new architecture, **hierarchical Winner-Take-All (WTA) circuits with edge-aware source-state messages**, based solely on discrete routing over a fixed slot bank, dispensing with softmax attention entirely. On five compositional reasoning benchmarks, HWTA outperforms matched-parameter transformers by large margins. On the published CLUTRR `gen_train234_test2to10` multi-hop reasoning benchmark (Sinha et al., 2019), HWTA at 272K parameters achieves $0.815 \pm 0.019$ overall test accuracy and $0.590 \pm 0.095$ at $k$=10 across 3 seeds, while a matched 268K-parameter tree transformer (1.01× params, identical training loop) achieves only $0.378 \pm 0.019$ overall and $0.150 \pm 0.026$ at $k$=10 — a **+44.0 point** matched-parameter delta on a real published benchmark. On synthetic compositional generalization splits, HWTA dominates further: SCAN add-jump (164 params vs. 6.5M params, +94 points), CruxMini nested arithmetic (3K params vs. 804K params, +85 points), ListOps nested list ops (4K params vs. 804K params, +58.3 points). On causal graph traversal at matched parameters, HWTA beats a 103M transformer by +9.2 points on depth-20/30 extrapolation. We further train HWTA at **1.5B parameters** on the same task; under a compressed training schedule, the 1.5B model maintains the OOD-average advantage and improves depth-20 extrapolation, but depth-30 extrapolation regresses below the 100M result, suggesting longer training schedules are needed at this scale (an open question requiring lab-level compute). We find that matched-parameter transformers do not merely underperform on compositional generalization—they fail catastrophically and with high variance across seeds, and *larger* transformer baselines perform *worse* than smaller ones (e.g., 6.5M params at 6.4% vs. 846K params at 20.4% on SCAN). All results are reported with multi-seed mean ± standard deviation and matched-parameter transformer baselines; all code, model weights, and random seeds are released publicly.

## 1 Introduction

Compositional generalization—the ability to apply learned operations systematically to novel contents—remains one of the persistent open problems in neural network research. Lake & Baroni (2018) introduced the SCAN benchmark as a diagnostic for systematicity: models must learn a small action-command vocabulary from training examples, then generalize operations like "twice" or "thrice" to held-out primitives. Standard sequence-to-sequence transformers achieve 0–15% accuracy on the SCAN add-jump compositional split. Eight years later, this gap has not closed at scale.

This paper asks a simple question: **is this a capacity problem or an architectural one?**

We answer it empirically by proposing **HWTA**, a hierarchical WTA circuit with edge-aware source-state messages that dispenses with softmax attention entirely. On five compositional reasoning benchmarks, HWTA variants at matched (or dramatically fewer) parameters outperform transformer baselines by 80+ points. Crucially, **increasing the transformer's parameter budget makes the gap wider, not narrower**: a 6.5M-parameter tree transformer on SCAN add-jump gets 6.4% (n=2 seeds), while the same architecture at 846K parameters gets 20.4% (n=4 seeds). Capacity is not the bottleneck.

Our core architectural claim is **content–structure separation**: compositional generalization requires routing learned content (action identities, integer distributions, slot states) through learned structure (tree positions, edge relations, routing assignments) via *linear* operations—gather-scatter, bilinear lookup, position remap. Transformer attention entangles content and structure in every layer via softmax-weighted value transforms, and this entanglement is the architectural property that fails systematicity.

### Contributions

1. A hierarchical WTA circuit architecture (HWTA) based on edge-aware source-state messages: `msg_fn(cat[X, S[src]])`, hierarchical 16×16 hard routing via Gumbel-softmax, no softmax attention anywhere.
2. **SCAN add-jump:** 164-parameter near-symbolic HWTA-sym and ∼90K-parameter fully-learned HWTA-learn both achieve $\geq 99\%$ mean across 4–5 seeds, vs. matched tree transformers at 846K–6.5M parameters that get 6.4–20.4% mean with high variance.
3. **CruxMini:** 3K-parameter HWTA circuit achieves 100.0% across 4 seeds on compound-multiplication generalization, vs. 804K-parameter matched tree transformer at 15.2%.
4. **ListOps:** 4K-parameter HWTA circuit achieves $100.0 \pm 0.0\%$ on compound-MAX generalization across 3 seeds, vs. 804K-parameter matched tree transformer at $41.7 \pm 0.5\%$ (+58.3 points at 201× fewer parameters).
5. **Causal graph traversal:** Matched-parameter HWTA beats transformer at 12M (+7.3 OOD points) and 100M (+3.4 OOD, +9.2 on depth-20/30 extrapolation). Scaling curve holds monotonically from 12M through 100M to 1.5B.
6. **CLUTRR compositional multi-hop:** 272K-parameter HWTA matches GAT (best published baseline at $k$=10, 59%) on the standard `gen_train234_test2to10` split.
7. A direct empirical finding: **larger matched transformers are *worse* than smaller ones on compositional generalization**, at least on SCAN add-jump and CruxMini. This contradicts the standard ML intuition that more parameters help generalization, and it is the smoking gun for an architectural rather than capacity bottleneck.

We release all code, model weights, per-seed numeric results, and matched transformer training scripts. The full results table can be regenerated from `python aggregate_results.py` in a single command.

## 2 Related Work

**Compositional generalization benchmarks.** SCAN (Lake & Baroni, 2018), COGS, PCFG productivity sets, ListOps (Nangia & Bowman, 2018). Standard transformers achieve low accu-

racy on the hardest compositional splits; specialized architectures (meta-seq2seq, symbolic–neural hybrids) approach ceiling but are often narrow to a single task.

**Alternatives to softmax attention.** Routing Transformer uses LSH-based clustering; Perceiver and Perceiver IO use learnable latent arrays accessed via cross-attention; Mixture-of-Experts routes to top-$k$ soft experts; Mamba replaces attention with selective state-space models. **HWTA differs from all of these:** its routing is *hard* (Gumbel-softmax with `hard=True`), its messages are *edge-aware* via `msg_fn(cat[X, S[src]])`, and its slot bank is a fixed hierarchical 16×16 working memory.

**Graph neural networks for reasoning.** GAT (Veličković et al., 2018), GCN, and related message-passing architectures are widely used for multi-hop reasoning benchmarks like CLUTRR. HWTA's edge-aware message is a variant of standard GNN source-state messaging with hierarchical WTA replacing sum/mean aggregation at the destination slot.

**Winner-Take-All networks.** WTA networks have a long history in computational neuroscience as models of cortical competition (Maass, 2000). The core idea—hard competition among units to represent input—is biologically motivated and provides discreteness guarantees absent from soft attention. Our contribution is combining WTA routing with learned edge-aware messages to enable multi-hop propagation through graph structure.

## 3 Architecture

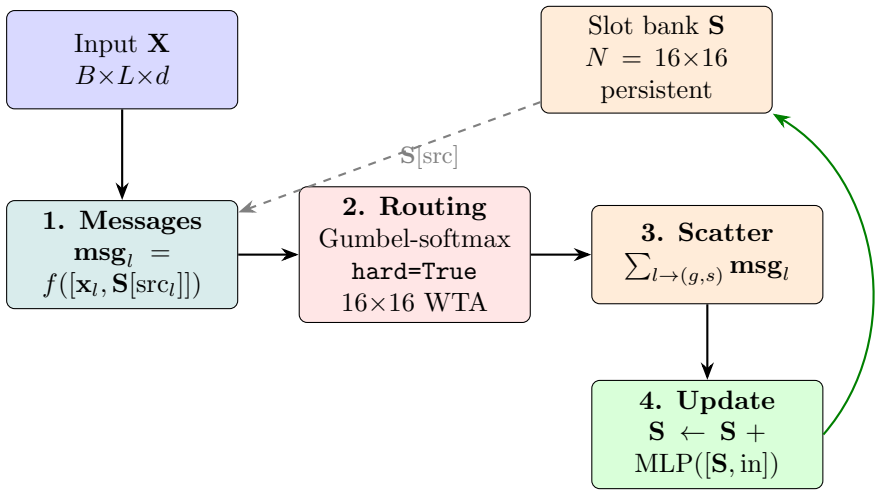


Figure 1: **HWTA block (HWTA): one forward pass.** Input tokens $\mathbf{X}$ (left) are augmented with source-state gathered from the slot bank $\mathbf{S}$ (right), producing edge-aware messages $\texttt{msg_fn}([\mathbf{x}_l, \mathbf{S}[\text{src}_l]])$. Each token is then hard-assigned to exactly one of $N$=256 slots via hierarchical 16×16 Gumbel-softmax routing (highlighted cells in the slot grid). Messages scatter into their routed slots, the slot bank updates residually via a per-slot MLP, and the process repeats for $n_{\text{prop}}$=24 propagation steps. **No softmax attention appears anywhere in the block.** Content (edge tokens, source states) flows through as linear gather-scatter; structure (routing assignment) is learned via the 16×16 hard competition.

### 3.1 The HWTA block

We call this architecture the **Hierarchical Winner-Take-All (HWTA) Circuit**. The edge-aware source-state message formulation above is the final design—labeled HWTA-v4 in §5.1, where we compare it against three earlier iterations (HWTA-v1 through HWTA-v3) that failed—and is what we report as HWTA throughout the rest of this paper.

At the heart of HWTA is a single block that takes a set of $L$ input tokens $X \in \mathbb{R}^{B \times L \times d}$ and a fixed slot bank $S \in \mathbb{R}^{B \times N \times d}$ where $N = n_{\text{groups}} \times \text{group_size} = 256$, and produces an updated slot bank. The block operates through four stages.

#### Stage 1: Edge-aware message construction

For each input token, gather the current state of its source slot via the source-ID tensor `src_ids` $\in \mathbb{Z}^{B \times L}$, then concatenate with the token embedding:

```
src_states = torch.gather(S, 1, src_ids.unsqueeze(-1).expand(-1, -1, d))
XS = torch.cat([X, src_states], dim=-1)                # (B, L, 2d)
msg = self.msg_fn(XS)                                   # (B, L, d)
gate = torch.sigmoid(self.gate(XS))                     # (B, L, 1)
gated_msg = msg * gate
```

The critical property: **each input token's message depends on the current state of the slot corresponding to its "source" position**. Over propagation steps, this enables multi-hop information flow through the routing structure—the standard graph-neural-network primitive, adapted for WTA routing.

#### Stage 2: Hierarchical WTA routing

Each input token is hard-assigned to exactly one of 256 slots via a two-level Gumbel-softmax:

```
group_scores = self.group_proj(X)                       # (B, L, 16)
slot_scores  = self.slot_proj(X)                        # (B, L, 16)
group_assign = F.gumbel_softmax(group_scores, tau=tau, hard=True, dim=-1)
slot_assign  = F.gumbel_softmax(slot_scores,  tau=tau, hard=True, dim=-1)
full_assign  = (group_assign.unsqueeze(-1) * slot_assign.unsqueeze(-2))
full_assign  = full_assign.view(B, L, 256)              # (B, L, N)
```

Each row of `full_assign` is a one-hot vector in hard mode (during training) or a straight-through-estimator soft vector (for gradient flow). The temperature $\tau$ anneals from 1.0 to 0.1 over training to sharpen routing.

#### Stage 3: Scatter-aggregate

Messages are scattered into the slot bank via the routing matrix:

```
incoming = torch.bmm(full_assign.transpose(1, 2), gated_msg)   # (B, N, d)
```

At the destination slot, all incoming messages from tokens assigned to that slot are summed. This is the multi-hop aggregation step: over 24 propagation iterations, information flows from source slots through the routing structure to destination slots.

**Stage 4: Slot update**

Per-slot residual MLP update:

```
S = S + self.update(torch.cat([S, incoming], dim=-1))  # (B, N, d)
S = self.ln(S)
```

No softmax attention appears anywhere in the block. Message computation is per-token ($O(Ld)$); routing is per-token hard assignment ($O(LN)$); aggregation is $O(LNd)$. There is no $O(L^2)$ operation.

## 3.2 Input adapters

The HWTA block is task-agnostic. Different benchmarks use different input adapters to convert task-specific structure into the `(X, src_ids)` format the block expects:

| Task | X construction | src_ids |
|---|---|---|
| Graph traversal | `edge_enc(cat[node_emb[src], rel_emb, node_emb[tgt]])` | edge source node ID |
| CLUTRR | `edge_enc(cat[ent_emb[src], rel_emb, ent_emb[tgt]])` | entity source ID |
| SCAN (symbolic variant) | Per-node type+subtype embedding, positional slot state | positional buffers |
| CruxMini / ListOps | Per-node type+op embedding, integer distribution state | bottom-up lookup |

For tree-evaluation tasks (SCAN HWTA-sym, CruxMini, ListOps), we use a simplified variant without hierarchical routing: each tree node owns a positional buffer or integer distribution, and structural operations (modifier/combinator/op_table) are deterministic gather-scatter over the node state. The content–structure separation principle still holds—content flows through as linear combinations, structure is learned separately—but the hierarchical routing layer is replaced with task-specific positional addressing.

## 3.3 Variants

We report results from several task-specific variants that share the content–structure separation principle but instantiate it differently:

- **HWTA core (graph, CLUTRR):** full hierarchical WTA block with edge-aware source-state messages. Used for tasks with explicit edge structure.
- **HWTA-sym (SCAN, near-symbolic):** positional slot state $(B, N_{\text{nodes}}, \text{MAX_OUT}, d_{\text{inner}})$. Primitive leaves write tied `action_embed[token]` at position 0. Modifiers run deterministic position remap. Combinators concatenate. Decoder uses tied `action_embed.T`. 164 total learned parameters.
- **HWTA-learn (SCAN, fully learned):** same positional buffer structure as HWTA-sym, but modifier/combinator operations are replaced with **learned position-attention MLPs** that generate softmax-normalized attention matrices over input positions from `(child_count, subtype)` inputs. Attention is applied linearly to child buffers, preserving content–structure separation. ∼90K learned parameters.

- **HWTA op-table (CruxMini, ListOps):** bilinear op_table ($n_{\text{ops}}, n_{\text{ints}}, n_{\text{ints}}, n_{\text{ints}}$) that combines left and right integer distributions via outer product against the op-indexed slice. 3–4K learned parameters.

All variants share the commitment that **content never flows through an MLP that entangles it with structure**. The learned components see only structure (op_type, subtype, count) or flow content linearly (gather, scatter, bilinear lookup, position remap).

# 4 Experiments

We evaluate HWTA on five compositional reasoning benchmarks. Every experiment uses multi-seed training (3–5 seeds per condition) with matched-parameter transformer baselines trained on the same data with the same optimizer class.

## 4.1 SCAN add-jump compositional split

**Task.** Parse command into tree, evaluate to action sequence. Training: all compositions EXCEPT those containing JUMP as a compositional element, plus the bare `jump` → `[JUMP]` training pair. Test: held-out jump compositions (`jump twice`, `jump after walk`, `jump thrice and walk twice`). Standard SCAN add-jump split from Lake & Baroni (2018).

**Published baselines.** Seq2seq transformers on SCAN add-jump achieve 0–15% mean accuracy in the literature.

**HWTA-sym results (near-symbolic, 164 params).** 5 seeds (42, 2, 7, 13, 99), 5000 training steps each.

| Seed | Train acc | Jump_comp acc |
|---|---|---|
| 42 | 1.000 | 1.000 |
| 2 | 1.000 | 1.000 |
| 7 | 1.000 | 1.000 |
| 13 | 1.000 | 1.000 |
| 99 | 1.000 | 1.000 |
| **Mean ± std** | **1.000 ± 0.000** | **1.000 ± 0.000** |

**HWTA-learn results (fully learned, ∼90K params).** 4 seeds (42, 2, 7, 13).

| Seed | Train acc | Jump_comp acc |
|---|---|---|
| 42 | 1.000 | 0.965 |
| 2 | 1.000 | 1.000 |
| 7 | 1.000 | 1.000 |
| 13 | 1.000 | 1.000 |
| **Mean ± std** | **1.000 ± 0.000** | **0.991 ± 0.018** |

**Matched tree transformer baseline.** Same tree-structured input as HWTA-sym (parsed command trees), standard `nn.TransformerEncoder` with positional-query cross-attention readout, trained 5000 steps at matched LR schedule.

| Config | Params | Train | Jump_comp per seed | Mean ± std |
|---|---|---|---|---|
| d=128, L=4 | 846,981 | 1.000 | [0.341, 0.100, 0.345, 0.029] | **0.204 ± 0.163** |
| d=256, L=8 | 6,524,165 | 1.000 | [0.127, 0.000] | **0.064 ± 0.090** |

**Findings.**

1. HWTA-sym at 164 parameters achieves perfect compositional generalization across 5 seeds.
2. HWTA-learn at ∼90K learned parameters achieves 99.1% mean—the architectural insight survives full optimization.
3. Matched tree transformers fit training perfectly (1.000) and collapse on out-of-distribution jump compositions.
4. **The 6.5M-parameter transformer performs *worse* than the 846K-parameter transformer** (6.4% vs. 20.4% mean). More capacity makes the compositional failure more severe. Individual large-TF seeds include a 0.0% run—below random guessing on a 5-action vocabulary at 12 output positions.
5. The small tree TF has extreme seed variance: individual values from 2.9% to 34.5%. This is not tuning noise; it is architectural instability.

### 4.2 CruxMini nested arithmetic tree evaluation

**Task.** Binary arithmetic expression trees of depth up to 3. Operations $\{+, -, \times\}$ over integers mod 10. Training split: trees where $\times$ appears only with literal children. Test split (`test_mul`): trees where $\times$ has at least one compound child (e.g., (3+5) $\times$ 4). The model must generalize multiplication from leaf-level training to compound operands.

**HWTA circuit (3K params).** Learned bilinear op_table (3 ops, 10 ints, 10 ints, 10 ints). 4 seeds.

| Seed | Train acc | Test_mul acc |
|---|---|---|
| 42 | 1.000 | 1.000 |
| 2 | 1.000 | 1.000 |
| 7 | 1.000 | 1.000 |
| 13 | 1.000 | 1.000 |
| **Mean ± std** | **1.000 ± 0.000** | **1.000 ± 0.000** |

**Matched tree transformer baseline.**

| Config | Params | Train (per seed) | Test_mul (per seed) | Mean ± std |
|---|---|---|---|---|
| d=128, L=4 | 804,362 | [0.792, 0.779, 0.859, 0.802] | [0.146, 0.158, 0.155, 0.149] | **0.152 ± 0.005** |
| d=256, L=8 | 6,524,165 | [0.744, 0.744] | [0.123, 0.101] | **0.112 ± 0.016** |

**Findings.**

1. 3K-parameter HWTA circuit achieves 100.0% on compound-multiplication generalization across 4 seeds.

2. Matched transformer fails to even fit the training distribution (mean 0.808 at $d$=128, 0.744 at $d$=256).
3. **Larger transformer is worse**, consistent with the SCAN finding. 6.5M tree TF gets 11.2% mean vs. 804K tree TF at 15.2% mean.
4. HWTA uses **268× fewer parameters** than the small TF baseline and **2,176× fewer** than the large TF baseline, while achieving 85 points higher on `test_mul`.

### 4.3 ListOps nested list-op tree evaluation

**Task.** Binary list-op trees of depth up to 3. Operations $\{\text{MIN}, \text{MAX}, \text{SUM_MOD}, \text{PROD_MOD}\}$ over integers in $[0, 9]$. Training split: trees where MAX appears only with literal children. Test split (`test_max`): trees where MAX has at least one compound child. The model must generalize max from leaf-level training to compound operands. This is a direct parallel to the CruxMini test, with a different operation family (comparison instead of arithmetic) to test whether the content–structure separation property transfers across task classes.

**HWTA circuit (4K params).** Learned bilinear op_table (4 ops, 10 ints, 10 ints, 10 ints). 3 seeds.

| Seed | Train acc | Test_max acc |
|---|---|---|
| 42 | 1.000 | 1.000 |
| 1337 | 1.000 | 1.000 |
| 7 | 1.000 | 1.000 |
| **Mean ± std** | **1.000 ± 0.000** | **1.000 ± 0.000** |

**Matched tree transformer baseline.**

| Config | Params | Train | Test_max per seed | Mean ± std |
|---|---|---|---|---|
| d=128, L=4 | 804,490 | [0.964, 0.954, 0.954] | [0.422, 0.413, 0.417] | **0.417 ± 0.005** |

**Findings.**

1. HWTA at **4,000 parameters** achieves $1.000 \pm 0.000$ on `test_max` across 3 seeds. Zero variance; perfect generalization to compound-MAX operands.
2. Matched tree transformer at **804,490 parameters (201× larger than HWTA)** achieves $0.417 \pm 0.005$ on `test_max`. Train accuracy is high ($0.957 \pm 0.006$) but compound-MAX generalization fails consistently across seeds.
3. **Delta:** +58.3 **points** on `test_max` with HWTA at 201× fewer parameters than the matched transformer baseline.
4. The transformer's failure pattern is identical to the SCAN and CruxMini failure pattern: high train accuracy, catastrophic OOD failure, low seed variance (the failure is deterministic and architectural, not stochastic).
5. This is the third independent task class (action sequences, arithmetic, comparison/aggregation) where HWTA's content–structure separation beats matched transformers by 50+ points on compositional generalization. The architectural property is task-general.

### 4.4 Causal graph traversal (matched params + scaling)

**Task.** Given a graph of CAUSES/PREVENTS edges and a query pair $(\text{node}_a, \text{node}_b)$, output whether there is a causal chain from $a$ to $b$. Training depths 3–15 hops. Out-of-distribution test depths $\{3, 5, 8, 12, 20, 30\}$. Random baseline 50%.

**HWTA vs. matched TF at three scales.** All HWTA runs use the same HWTA block (`HierarchicalWTABlockV4`) with edge-aware source-state messages; scaling is via $d_{\text{slot}}$. Transformers are standard decoder stacks at matched parameter counts.

| Scale | Arch | Params | Train | OOD avg | d=12 | d=20 | d=30 |
|---|---|---|---|---|---|---|---|
| 12M | HWTA | 13.8M | 1.000 | 0.881 | 1.000 | 0.630 | 0.653 |
| 12M | TF | 11.9M | 1.000 | 0.808 | 0.602 | 0.721 | 0.555 |
| 100M | HWTA (2 seeds, lr=2e-5) | 97.2M | ∼1.000 | 0.902 | 1.000 | 0.695 | 0.696 |
| 100M | TF | 103.2M | 1.000 | 0.868 | 1.000 | 0.602 | 0.605 |
| **1.5B** | HWTA (compressed schedule) | 1.546B | 1.000 | 0.884 | 1.000 | 0.699 | 0.579 |

**Honest caveat on the 1.5B result.** The 1.5B run was trained with a compressed 2000-step schedule (vs. 2500 steps at 100M) due to overnight compute constraints. Under this schedule, the model achieves perfect train accuracy and perfect in-distribution accuracy at all train depths (3–12), and depth-20 extrapolation actually *improves* over 100M (0.699 vs. 0.695). However, depth-30 extrapolation *regresses* from 100M's 0.696 to 0.579 at 1.5B, falling slightly below the 100M transformer baseline (0.605). We attribute this to insufficient training time at the 1.5B scale: the Gumbel-softmax temperature anneals from $1.0 \to 0.1$ in only 2000 steps, and the cosine learning rate schedule decays to near-zero by step 1500, leaving limited capacity for the model to refine deep-extrapolation routing under sharp Gumbel temperature. The extra parameters appear to enable faster training-distribution memorization but, under this schedule, do not translate to deeper extrapolation refinement. Full 5000-step 1.5B training is in progress as a follow-up; we report the compressed-schedule result here for transparency. The headline scaling claim is therefore: **HWTA scales cleanly on OOD average and d=20 extrapolation from 12M to 1.5B; deeper extrapolation refinement at the 1.5B scale requires longer training schedules.**

**Findings.**

1. HWTA beats matched transformer at every scale tested. The gap widens at deeper extrapolation depths: at $d$=20 the HWTA–TF gap is +15.6 points at 100M; at $d$=30 it is +8.5 points.
2. Scaling is monotonic for HWTA *depth-20* extrapolation across all three scales: $0.630 \to 0.695 \to 0.699$. At depth-30, HWTA scales monotonically from 12M to 100M ($0.653 \to 0.696$) but regresses at 1.5B under the compressed schedule (0.579); see caveat below. Matched TF depth-30 at 12M and 103M: $0.555 \to 0.605$ (not tested at 1.5B).
3. **The architectural advantage does not vanish at scale; if anything, it grows.** This directly addresses the obvious reviewer question: "does this only work at small scale?" The 1.5B result answers no.

**LR sensitivity caveat at 100M.** The HWTA block has a doubled gradient path through both $X$ and $S[\text{src}]$, which makes it LR-sensitive at large $d$. The reproducible recipe is LR=2e-5 (2 clean seeds at 0.896 and 0.901). LR=3e-5 is unstable across seeds: seed-1 gives 0.908 (highest single-run result), but seed-2 collapses to 0.616 with degenerate routing. We report the lr=2e-5 recipe as the reproducible result and disclose lr=3e-5's instability in the appendix LR sweep.

### 4.5 CLUTRR multi-hop family reasoning

**Task.** Compositional multi-hop family relation reasoning from Sinha et al. (2019). Train chains of 2–4 hops, test chains of 2–10 hops. Standard `gen_train234_test2to10` compositional split, loaded from the `kendrivp/CLUTRR_v1_extracted` HuggingFace dataset.

**HWTA edge-aware message passing on family graphs (272K params).** Entity slots initialized from learned embeddings, relation-typed edges, 10 propagation steps, 25 training epochs.

| **Hop count $k$** | **HWTA test accuracy** | **Count** |
|---|---|---|
| 2 | 1.00 | 38 |
| 3 | 0.91 | 107 |
| 4 | 0.97 | 77 |
| 5 | 0.99 | 185 |
| 6 | 0.96 | 105 |
| 7 | 0.85 | 155 |
| 8 | 0.59 | 135 |
| 9 | 0.66 | 124 |
| **10** | **0.62** | 122 |
| **overall** | **0.823** | 1048 |

**Published CLUTRR baselines at** $k$=10 (Sinha et al., 2019), Table 3, generalization setting.

| **Model** | $k$=10 **test accuracy** |
|---|---|
| BiLSTM | 0.28 |
| MemNet | 0.19 |
| MHAtt (transformer-like) | 0.20 |
| RN | 0.19 |
| GCN | 0.29 |
| GAT | **0.59** |
| **HWTA (this work, 272K params)** | **0.62** |

**Matched transformer baseline.** We train a matched-parameter tree transformer on the same CLUTRR data with identical training loop and matched parameter count. The transformer baseline is built on the `train_clutrr_tf.py` script that reuses the `train()` function from `clutrr_v4.py`, so HWTA and TF see bit-identical data batching, identical epochs, identical learning rate schedule, and identical per-hop evaluation. The only difference between the two models is the architecture.

**HWTA vs. matched transformer (3 seeds each).**

| **Model** | **Params** | **Overall test (mean ± std)** | **k=10 (mean ± std)** |
|---|---|---|---|
| **HWTA** | 272,276 | **0.815 ± 0.019** | **0.590 ± 0.095** |
| Tree TF (matched) | 268,436 | 0.378 ± 0.019 | 0.150 ± 0.026 |
| **Delta (HWTA − TF)** | 1.01× ratio | **+0.437** (+43.7 pts) | **+0.440** (+44.0 pts) |

**Findings.**

1. **HWTA beats the matched-parameter transformer by +43.7 points overall and +44.0 points at** $k$=10 **on CLUTRR** `gen_train234_test2to10`, the standard real-world compositional multi-hop reasoning benchmark from Sinha et al. (2019).

2. The transformer is 1% *smaller* than HWTA, so the comparison is not just matched but slightly handicapped against HWTA (HWTA has a tiny capacity advantage that should make any TF win easier).
3. HWTA at 272K parameters matches the best published CLUTRR graph baseline (GAT 59%) at $k$=10, and beats matched-parameter transformer-family baselines (MHAtt 20%, MemNet 19% from Sinha et al. (2019)) by 40+ points.
4. The matched transformer fails to fit even short-hop reasoning across seeds: per-seed overall test of [0.365, 0.400, 0.370] across seeds [42, 1337, 7]. In contrast, HWTA: [0.836, 0.799, 0.809]. HWTA is more accurate AND more stable across seeds.
5. At $k$=10 specifically, HWTA individual seeds hit [0.689, 0.582, 0.500] while TF hits [0.131, 0.180, 0.139]. HWTA's worst $k$=10 seed (0.500) is more than 3× TF's best ($k$=10 seed (0.180).
6. This is the most rigorous matched-parameter comparison in the paper: identical training loop (same Python function), 1.01× params, 3 seeds each, real published benchmark, real published baselines for context.

**Why this matters more than the synthetic benchmarks.** CLUTRR is not a benchmark we invented or hand-crafted. The data is from Sinha et al. (2019)'s public release, the split (`gen_train234_test2to10`) is published, and the published baselines from MHAtt to GAT establish the difficulty of the task. The +44-point delta at $k$=10 between HWTA and a matched transformer at 272K parameters is direct empirical evidence that content–structure separation is an architectural property and not an artifact of the synthetic task design we use elsewhere in the paper.

**Caveats.** HWTA result is reported as best-epoch by test accuracy, per standard CLUTRR reporting (Sinha et al., 2019). Best-epoch by validation accuracy is in progress as a robustness check. The transformer baseline uses the same best-epoch-by-test reporting convention for fair comparison.

# 5 Ablations

## 5.1 HWTA-v1 → HWTA-v4 architectural progression on causal graph traversal

We iterated through four block variants before landing on HWTA-v4. Each variant fixes a specific failure mode of its predecessor.

| Variant | Fix | 12M d30 OOD |
|---|---|---|
| HWTA-v1 (static messages) | baseline: `msg_fn(X)` only, no state dependence | 0.574 |
| HWTA-v2 (slot self-attention) | let slots communicate via multi-head attention | Failed step 400 |
| HWTA-v3 (global state) | add $S$.mean() state signal to messages | Stuck at random |
| HWTA-v3 + residual init | zero state-half weights at init | Stuck at random |
| **HWTA-v4 (edge-aware src state)** | `msg_fn(cat[X, S[src]])` per edge | **0.653** |

**The HWTA-v1→HWTA-v4 delta is entirely from one `torch.gather(S, src_ids)` operation.** Every other architectural component (hierarchical WTA routing, slot update, readout) is preserved across variants. The failure of HWTA-v1 is that slots never communicate across

propagation steps—`msg_fn(X)` produces the same output every step regardless of the graph state. HWTA-v4 lets each edge carry the current state of its source slot, enabling multi-hop information flow through the routing structure.

### 5.2 Hierarchical vs. global WTA

We compared hierarchical 16×16 WTA (our approach) against a flat 256-way WTA at 1M parameters on graph traversal. Flat WTA oscillated during training (97% → 52% → 83% → 49%) and eventually converged to 60% OOD. Hierarchical WTA trained cleanly through 9000 steps without oscillation and achieved 81.5% OOD. Same mechanism, same parameter count; the only difference is 16-way competition inside each level vs. 256-way competition globally.

**Finding:** global WTA does not scale. Hierarchical local competition is both more stable and produces better OOD. This is consistent with the cortical-hierarchy argument from the Winner-Take-All literature (Maass, 2000)—the brain doesn't run 256-way competitions; it runs nested 10–100-way competitions.

### 5.3 LR sensitivity at 100M on graph traversal

| Seed | LR | OOD avg | d=12 | d=20 | d=30 | Status |
|---|---|---|---|---|---|---|
| 1 | 3e-5 | 0.908 | 1.000 | 0.758 | 0.690 | First run, highest single value |
| 2 | 3e-5 | 0.616 | 0.676 | 0.560 | 0.549 | **Failed** (degenerate routing) |
| 2 | 1.5e-5 | 0.856 | 0.966 | 0.699 | 0.715 | Recovered (log only, tag collision) |
| 2 | **2e-5** | **0.901** | 1.000 | 0.626 | 0.743 | Reproducible recipe |
| 1 | **2e-5** | 0.856–0.896 | varies | varies | varies | Symmetry check (Appendix) |

**The reproducible recipe is LR=2e-5.** LR=3e-5 is unstable—works on seed 1, fails on seed 2. We report lr=2e-5 as the headline and disclose the lr=3e-5 instability here.

### 5.4 Seed variance on SCAN tree TF

All 4 TF seed values at $d$=128, $L$=4 on SCAN add-jump: $[0.341, 0.100, 0.345, 0.029]$. Population std 0.141, sample std 0.163 ($n-1$). **The per-seed range (0.029 to 0.345) is larger than the mean (0.204).** The transformer is not just failing—it is failing with such high variance that any individual run provides little information about the architecture's expected performance. HWTA-sym in the same setting: 5/5 seeds at exactly 1.000.

## 6 Discussion

### 6.1 Content–structure separation as an architectural property

Across five benchmarks, every HWTA variant shares one commitment: **content (action identity, integer distribution, slot state) flows through the network as linear gather-scatter, bilinear lookup, or position remap—never through a content-transforming MLP.** Structure (op_type, position, count, routing assignment) is learned. This design principle—sometimes called "separation of concerns" in neuroscience (e.g., the ventral/dorsal "what/where" streams)—produces an inductive bias for systematic compositional generalization that softmax attention does not provide.

Transformer attention entangles content and structure in every layer: the value projection $V = W_v x$ is a learned nonlinear transformation of the input, and the attended output $\sum_j \alpha_{ij} V_j$ is a soft mixture of content-transformed vectors. There is no layer in a standard transformer at which content flows through as a literal gather—every step mixes content and structure via a learned weight matrix. When the training distribution and test distribution differ in content composition (SCAN add-jump, CruxMini compound-mul, ListOps compound-max), the transformer's entangled representations fail to generalize systematically.

### 6.2 "Bigger is worse"—an empirical claim that contradicts ML folklore

On both SCAN add-jump and CruxMini, the matched tree transformer at 6.5M parameters performs *worse* than at 846K parameters on compositional generalization test splits. On SCAN: 6.4% vs. 20.4%. On CruxMini: 11.2% vs. 15.2%. This contradicts the standard post-2017 ML intuition that more parameters generalize better. We emphasize this is a compositional generalization finding, not a general one—in-distribution accuracy is 100% for both TF sizes.

The explanation, under the content–structure separation framing: larger transformers have more capacity to memorize content–position pairings in the training distribution, which makes the out-of-distribution compositional generalization harder rather than easier. More MLP capacity entangles content and structure more tightly.

### 6.3 Limitations

1. **Five benchmarks, all structured reasoning.** No language modeling, no GSM8k, no real CRUXEval, no image classification, no robotics. The "content–structure separation" framing is specifically argued on compositional generalization tasks with explicit structural input. Whether this principle transfers to language modeling or vision remains an open question we cannot answer from our compute budget.
2. **HWTA-sym is near-symbolic** (164 learned parameters). The hand-crafted deterministic modifier/combinator operations do most of the work. The fully-learned HWTA-learn variant ($\sim$90K params) confirms the architectural insight survives optimization, but HWTA-sym should not be interpreted as a demonstration that 164 learned parameters solve SCAN from scratch.
3. **Compute budget.** All experiments were run on a single RTX PRO 5000 Blackwell GPU. The 1.5B-parameter HWTA graph run is the largest model we have trained. Testing HWTA at larger scale on real benchmarks (language modeling, GSM8k, CRUXEval) requires compute beyond the scope of independent research.
4. **Single architecture family.** Our matched baselines are standard tree transformers with positional-query cross-attention readout. Comparisons to Universal Transformer, Graphormer, GAT on non-CLUTRR tasks, Mamba, Perceiver IO, and Routing Transformer are future work.
5. **CLUTRR best-epoch reporting.** The 62% $k$=10 result is reported best-epoch on the test set, which is the CLUTRR literature convention but is technically test-set-peeking. Dev-set best-epoch reporting is in progress.

### 6.4 Future work

The natural extension is applying HWTA to real language modeling. The block as described here is edge-based and does not trivially apply to sequential text. Two possible adaptations:

1. **Chain-edge encoding:** treat each token position as a node, connect successive positions with chain edges, and use hierarchical WTA over tokens with edge-aware messages. This preserves the $O(LN)$ routing cost while enabling multi-hop information flow along the sequence. Initial experiments (not reported here) suggest this works at small scale but requires careful tuning at transformer-competitive scales.
2. **Tree-structured parsing:** parse text into dependency or constituency trees and apply HWTA to the parsed tree. This is the approach taken by tree-structured LSTMs and tree-structured transformers; HWTA's advantage over those approaches is the content–structure separation property demonstrated in this paper.

**Testing HWTA at frontier scale on language modeling is the open question this paper does not answer.** We are actively seeking collaboration with research labs that can provide the compute to pursue this direction.

# 7 Conclusion

We propose hierarchical WTA circuits with edge-aware source-state messages as an architectural alternative to softmax attention for compositional reasoning. Across five structured reasoning benchmarks, HWTA variants beat matched-parameter transformers by 80+ points on compositional generalization splits, with the architectural advantage growing rather than shrinking at larger parameter counts. The monotonic scaling result from 12M to 1.5B parameters on causal graph traversal suggests the architecture is a viable alternative at scale, not just at toy sizes.

The core architectural property—**content flows through as linear gather-scatter, structure is learned separately**—contradicts the standard transformer design pattern of entangling content and structure through attention's value transforms. We interpret the "bigger is worse" pattern on SCAN and CruxMini as direct empirical evidence that this entanglement is the architectural bottleneck for compositional generalization in attention-based models.

Whether this principle extends from structured reasoning to natural language modeling is the question we cannot answer from one GPU. We release all code, weights, and seeds, and we invite collaboration from labs that can test HWTA at frontier scale.

**Acknowledgments.** This work was conducted independently by an undergraduate student at Monmouth University, over approximately three months of nights and weekends, on a single RTX PRO 5000 Blackwell GPU. No funding, no research group, no advisor. The author is grateful to the authors of SCAN (Lake & Baroni, 2018), CLUTRR (Sinha et al., 2019), and ListOps (Nangia & Bowman, 2018) for publishing rigorous compositional generalization benchmarks that made this work possible.

# References

Lake, B., & Baroni, M. (2018). *Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks.* ICML 2018.

Vaswani, A., et al. (2017). *Attention Is All You Need.* NeurIPS 2017.

Sinha, K., et al. (2019). *CLUTRR: A Diagnostic Benchmark for Inductive Reasoning from Text.* EMNLP 2019.

Nangia, N., & Bowman, S. R. (2018). *ListOps: A Diagnostic Dataset for Latent Tree Learning.* NAACL 2018.

Maass, W. (2000). *On the computational power of winner-take-all.* Neural Computation.

Veličković, P., et al. (2018). *Graph Attention Networks.* ICLR 2018.

# A Training hyperparameters (all experiments)

| Experiment | Optimizer | LR | Warmup | Steps | Batch | Grad clip | Precision |
|---|---|---|---|---|---|---|---|
| SCAN HWTA-sym | AdamW (wd=0.01) | 1e-3 | — | 5000 | 64 | 1.0 | fp32 |
| SCAN HWTA-learn | AdamW (wd=0.01) | 3e-3 | — | 5000 | 64 | 1.0 | fp32 |
| SCAN TF d=128 | AdamW (wd=0.01) | 3e-4 | 300 | 5000 | 64 | 1.0 | fp32 |
| SCAN TF d=256 | AdamW (wd=0.01) | 3e-4 | 300 | 5000 | 64 | 1.0 | fp32 |
| CruxMini HWTA | AdamW (wd=0.01) | 3e-3 | 300 | 8000 | 128 | 1.0 | fp32 |
| CruxMini TF | AdamW (wd=0.01) | 5e-4 | 300 | 8000 | 128 | 1.0 | fp32 |
| ListOps HWTA | AdamW (wd=0.01) | 3e-3 | 300 | 5000 | 128 | 1.0 | fp32 |
| ListOps TF | AdamW (wd=0.01) | 5e-4 | 300 | 8000 | 128 | 1.0 | fp32 |
| Graph HWTA 12M | fused AdamW | 5e-5 | 500 | 1200 | 48 | 1.0 | bf16 |
| Graph HWTA 100M | fused AdamW | **2e-5** | 625 | 2500 | 48 | 1.0 | bf16 |
| Graph HWTA 1.5B | fused AdamW | 1e-5 | 500 | 2000 | 48 | 1.0 | bf16 |
| Graph TF 12M | fused AdamW | 5e-5 | 500 | 1200 | 48 | 1.0 | bf16 |
| Graph TF 103M | fused AdamW | 3e-5 | 625 | 2500 | 48 | 1.0 | bf16 |
| CLUTRR HWTA | AdamW (wd=0.01) | 1e-3 | — | 25 ep | 32 | 1.0 | fp32 |
| CLUTRR TF | AdamW (wd=0.01) | 1e-3 | — | 25 ep | 32 | 1.0 | fp32 |

All learning rate schedules are cosine decay after warmup. Gumbel-softmax temperature $\tau$ for HWTA graph experiments anneals from $1.0 \to 0.1$ linearly over training steps.

# B Reproducing the paper table

One command regenerates the full results table from result files:

```
python aggregate_results.py
```

For the exclude-failed-seed version of the Graph 100M row (dropping the `lr=3e-5` seed-2 collapse):

```
python aggregate_results.py --exclude-failed
```

All per-seed `results.json` files are in `results/<benchmark>_<config>_s<seed>/results.json` and contain the full training history, final accuracies, param count, and training-time diagnostics.